NOI2022 省选

DAY1

时间：2023 年 4 月 1 日 08:30 ～ 2022 年 4 月 1 日 13:00

题目名称	火车站	城市建造	人员调度
题目类型	传统型	传统型	传统型
目录	`station`	`cities`	`transfer`
可执行文件名	`station`	`cities`	`transfer`
输入文件名	`station.in`	`cities.in`	`transfer.in`
输出文件名	`station.out`	`cities.out`	`transfer.out`
每个测试点时限	1.0 秒	1.0 秒	5.0 秒
内存限制	512 MiB	512 MiB	512 MiB
测试点数目	10	20	50
测试点是否等分	是	是	是

提交源程序文件名

对于 C++ 语言	`station.cpp`	`cities.cpp`	`transfer.cpp`

编译选项

对于 C++ 语言	`-O2 -std=c++14 -static`

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考各省的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 全国统一评测时采用的机器配置为：Intel(R) Core(TM) i7-8700K CPU @3.70GHz，内存 32GB。上述时限以此配置为准。
9. 只提供 Linux 格式附加样例文件。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

火车站（station）

【题目描述】

有 n 个火车站排成一条直线，从 1 到 n 编号。一共有 m 条火车轨道，每条轨道覆盖一段火车站区间 $[l_i, r_i]$。

对于一个被多条火车轨道覆盖的火车站，火车在经过这里的时候，可以在此处改变轨道。但是火车无法掉头，只能朝着一个方向运行（即只能一直往 1 的方向开或者一直往 n 的方向开）。

小 A 从火车站 x 出发，即搭上了经过 x 的任意一列火车（这列火车也可能是从车站 x 出发）。这列火车可能行驶在火车站 x 所处的任一条轨道上，其运行方向既可能是往 1 的方向开，也可能是往 n 的方向开。小 A 上车后就开始昏睡，直到乘坐的火车到达某条线路的终点站停下，他才醒过来。问小 A 最后可能到达的车站。

注意：火车应运行至少一个车站，且火车切换轨道后不会立刻停下来，而是会继续沿着当前轨道前进。

【输入格式】

从文件 ***station.in*** 中读入数据。

输入的第一行包含三个正整数 n, m, x，分别表示火车站的数量，火车轨道的数量以及小 A 初始的起点。

接下来 m 行，每行包含两个正整数 l_i, r_i，表示一条火车轨道运行的区间。

【输出格式】

输出到文件 ***station.out*** 中。

输出一行，包含若干个用单个空格分隔的正整数，表示小 A 最后可能到达的车站，按照车站编号升序排序输出。

【样例 1 输入】

```
7 5 4
3 4
4 6
1 3
5 7
4 6
```

【样例 1 输出】

```
1 3 6 7
```

【样例 1 解释】

火车从车站 4 出发，沿着第一条轨道可以运行到终点 3，也可以接着沿第三条轨道运行到终点 1。

火车从车站 4 出发，沿着第二条轨道可以运行到终点 6，也可以在车站 5 换到第四条轨道运行到终点 7。

所以最终按顺序输出 1, 3, 6, 7。

【样例 2】

见选手目录下的 ***station/station2.in*** 与 ***station/station2.ans***。

【样例 3】

见选手目录下的 ***station/station3.in*** 与 ***station/station3.ans***。

【样例 4】

见选手目录下的 ***station/station4.in*** 与 ***station/station4.ans***。

【数据范围】

对于所有的数据，保证 $1 \le n, m \le 2 \times 10^5$；$1 \le x \le n$；$1 \le l_i < r_i \le n$。

测试点	$n, m \le$	特殊性质
1	50	无
2		
3	5000	
4		
5		
6	2×10^5	A
7		
8		无
9		
10		

特殊性质 A：保证 $x = 1$。

城市建造（cities）

【题目描述】

在这个国度里面有 n 座城市，一开始城市之间修有若干条双向道路，导致这些城市形成了 $t \geq 2$ 个连通块，特别的，这些连通块之间两两大小差的绝对值不超过 $0 \leq k \leq 1$。为了方便城市建设与发展，n 座城市中的某 t 座城市**在这 t 座城市之间**额外修建了至少一条双向道路，使得所有城市连通。

现在已经知道额外修建后的所有道路，你需要算出有哪些双向道路集合 E'，满足这些道路有可能是后来额外修建的，请输出答案对 $998,244,353$ 取模的结果。

即给定一张 n 个点 m 条边的**无向连通**图 $G = (V, E)$，询问有多少该图的子图 $G' = (V', E')$，满足 $E' \neq \varnothing$ 且 $G - E'$ 中恰好有 $|V'|$ 个连通块，且任意两个连通块大小之差不超过 k，保证 $0 \leq k \leq 1$，请输出答案对 $998,244,353$ 取模的结果。

【输入格式】

从文件 ***cities.in*** 中读入数据。

输入的第一行包含三个正整数 n, m, k，分别表示城市数、修建后的道路数以及任意两个连通块大小之差的上限。

接下来 m 行每行包含两个正整数 u, v，表示城市 u 和 v 之间存在一条双向道路，保证 $u \neq v$。

【输出格式】

输出到文件 ***cities.out*** 中。

输出一个数表示答案对 $998,244,353$ 取模后的结果。

【样例 1 输入】

```
4 4 1
1 2
2 3
1 3
3 4
```

【样例 1 输出】

```
2
```

【样例 1 解释】

有以下两种情况：

- 本来只有 $(3,4)$ 这一条道路，此时有三个连通块，分别为 $\{1\},\{2\},\{3,4\}$；后来城市 $1,2,3$ 决定在他们三座城市中额外修建了 $(1,2),(2,3),(1,3)$ 这三条道路，使得所有城市连通。
- 本来没有任何道路，此时有四个连通块，分别为 $\{1\},\{2\},\{3\},\{4\}$；后来城市 $1,2,3,4$ 决定在他们四座城市中额外修建了 $(1,2),(2,3),(1,3),(3,4)$ 这四条道路，使得所有城市连通。

【样例 2】

见选手目录下的 ***cities/cities2.in*** 与 ***cities/cities2.ans***。

【样例 3】

见选手目录下的 ***cities/cities3.in*** 与 ***cities/cities3.ans***。

【样例 4】

见选手目录下的 ***cities/cities4.in*** 与 ***cities/cities4.ans***。

【数据范围】

对于所有的数据，保证：$3 \le n \le 10^5$; $n-1 \le m \le 2\times 10^5$; $0 \le k \le 1$。

<table>
<tr><th>测试点</th><th>n</th><th>m</th><th>k</th></tr>
<tr><td>$1,2$</td><td>≤ 15</td><td>≤ 20</td><td>$=0$</td></tr>
<tr><td>$3\sim 5$</td><td>≤ 20</td><td>≤ 50</td><td>$=1$</td></tr>
<tr><td>$6,7$</td><td>≤ 200</td><td>≤ 300</td><td>$=0$</td></tr>
<tr><td>$8,9$</td><td rowspan="3">$\le 2,000$</td><td>$=n-1$</td><td>$=1$</td></tr>
<tr><td>$10,11$</td><td rowspan="2">$\le 3,000$</td><td>$=0$</td></tr>
<tr><td>$12,13$</td><td rowspan="2">$=1$</td></tr>
<tr><td>$14,15$</td><td rowspan="3">$\le 10^5$</td><td>$=n-1$</td></tr>
<tr><td>$16,17$</td><td rowspan="2">$\le 2\times 10^5$</td><td>$=0$</td></tr>
<tr><td>$18\sim 20$</td><td>$=1$</td></tr>
</table>

人员调度（transfer）

【题目描述】

众所周知，一个公司的 n 个部门可以组织成一个树形结构。形式化地，假设这些部门依次编号为 $1,...,n$，那么除了 1 号部门以外，第 $i \in [2,n]$ 个部门**有且仅有**一个上级部门 $p_i \in [1,i-1]$。这样，这家公司的 n 个部门可以视为一个以 1 为根的树。如果 i 是 j 子树中的点，那么称部门 i 是部门 j 的子部门。

该公司初始时有 k 名优秀员工，编号依次为 $1 \ldots k$。第 i 名优秀员工初始时在第 x_i 个部门工作，并且其有一个能力值 $v_i > 0$。

为了最大化公司的运作效率，公司老板 0/\/\G 决定进行一些人员调动。具体来说，可以将编号为 i 的优秀员工调动到 x_i 的一个子部门，或者不调度（此时该员工在 x_i 部门）。随后，优秀员工们会在其所在的部门竞选部门领导——能力值最高者将担任这一职位，并给公司带来等同于其能力值的贡献。如果一个部门一个优秀员工也没有，那么就无法选出部门领导，从而对公司的贡献将是 0。此时，公司的业绩被定义为公司各部门的贡献之和。

公司老板 0/\/\G 自然想知道，该如何进行人员调动，使公司的业绩最大？

这当然难不倒他，然而，公司优秀员工的数量也会发生变化；具体来说，会依次发生 m 个事件，每个事件形如：

1 x v：先令 $k = k + 1$，然后新增一位编号为 k、初始部门为 x、能力值为 v 的优秀员工；

2 id：编号为 id 的优秀员工将被辞退。

公司老板 0/\/\G 希望你能在最开始和每个事件发生后，告诉他公司的业绩最大可能是多少？

注意，每次人员调动都是独立的，也就是每次计算公司的最大可能业绩时，每个优秀员工都会回到其所在的初始部门。

【输入格式】

从文件 ***transfer.in*** 中读入数据。

输入的第一行包含一个正整数 sid，表示该测试点对应的数据范围以及特殊性质，详见后表；

输入的第二行包含三个整数 n, k, m，分别表示部门数，初始优秀员工数和事件数。

输入的第三行包含 $n-1$ 个正整数 $p_2, ..., p_n$，表示每个部门的上级部门。

接下来 k 行，每行包含两个正整数 x_i, v_i，表示优秀员工的初始部门和能力值。

接下来 m 行，每行形如 1 x v 或 2 id 表示一次事件。

【输出格式】

输出到文件 ***transfer.out*** 中。

输出一行包含 $m+1$ 个由单个空格隔开的非负整数，依次表示最开始和每个事件发生后，公司的业绩可能的最大值。

【样例 1 输入】

```
1
3 2 1
1 1
2 1
1 3
1 2 2
```

【样例 1 输出】

```
4 5
```

【样例 2】

见选手目录下的 ***transfer/transfer2.in*** 与 ***transfer/transfer2.ans***。

【样例 3】

见选手目录下的 ***transfer/transfer3.in*** 与 ***transfer/transfer3.ans***。

【样例 4】

见选手目录下的 ***transfer/transfer4.in*** 与 ***transfer/transfer4.ans***。

【样例 5】

见选手目录下的 ***transfer/transfer5.in*** 与 ***transfer/transfer5.ans***。

【样例 6】

见选手目录下的 ***transfer/transfer6.in*** 与 ***transfer/transfer6.ans***。

【样例 7】

见选手目录下的 ***transfer/transfer7.in*** 与 ***transfer/transfer7.ans***。

【样例 8】

见选手目录下的 ***transfer/transfer8.in*** 与 ***transfer/transfer8.ans***。

【样例 9】

见选手目录下的 ***transfer/transfer9.in*** 与 ***transfer/transfer9.ans***。

【样例 10】

见选手目录下的 ***transfer/transfer10.in*** 与 ***transfer/transfer10.ans***。

【样例 11】

见选手目录下的 ***transfer/transfer11.in*** 与 ***transfer/transfer11.ans***。

【样例 12】

见选手目录下的 ***transfer/transfer12.in*** 与 ***transfer/transfer12.ans***。

【样例 13】

见选手目录下的 ***transfer/transfer13.in*** 与 ***transfer/transfer13.ans***。

【样例 14】

见选手目录下的 ***transfer/transfer14.in*** 与 ***transfer/transfer14.ans***。

【样例 15】

见选手目录下的 ***transfer/transfer15.in*** 与 ***transfer/transfer15.ans***。

【数据范围】

对于所有的数据，保证：$1 \le sid \le 15$；$1 \le n, k \le 10^5$；$0 \le m \le 10^5$；$1 \le p_i < i$；$1 \le x_i, x \le n$；$1 \le v_i, v \le 10^5$；

对于事件 2，保证：$1 \le id \le k$ 且编号为 id 的员工在此事件发生仍在工作。

<table>
<tr><th>测试点编号</th><th>sid</th><th>$n \le$</th><th>$k \le$</th><th>$m \le$</th><th>特殊性质</th></tr>
<tr><td>1</td><td>1</td><td>6</td><td rowspan="2">6</td><td rowspan="2">6</td><td rowspan="5">无</td></tr>
<tr><td>2,3</td><td>2</td><td>9</td></tr>
<tr><td>4,5</td><td>3</td><td>16</td><td rowspan="2">66</td><td>66</td></tr>
<tr><td>$6 \sim 8$</td><td>4</td><td>66</td><td rowspan="4">0</td></tr>
<tr><td>$9 \sim 11$</td><td>5</td><td>$2,333$</td><td>$2,333$</td></tr>
<tr><td>$12 \sim 14$</td><td>6</td><td rowspan="2">10^5</td><td rowspan="2">10^5</td><td>B</td></tr>
<tr><td>$15 \sim 18$</td><td>7</td><td>无</td></tr>
<tr><td>$19 \sim 21$</td><td>8</td><td>$2,333$</td><td>$2,333$</td><td>$2,333$</td><td>A</td></tr>
<tr><td>$22 \sim 24$</td><td>9</td><td rowspan="2">10^5</td><td rowspan="2">10^5</td><td rowspan="2">10^5</td><td>AB</td></tr>
<tr><td>$25 \sim 28$</td><td>10</td><td>A</td></tr>
<tr><td>$29 \sim 31$</td><td>11</td><td>$2,333$</td><td>$2,333$</td><td>$2,333$</td><td>无</td></tr>
<tr><td>$32 \sim 34$</td><td>12</td><td rowspan="2">10^5</td><td rowspan="2">10^5</td><td rowspan="2">10^5</td><td>C</td></tr>
<tr><td>$35 \sim 38$</td><td>13</td><td>B</td></tr>
<tr><td>$39 \sim 44$</td><td>14</td><td>$66,666$</td><td>$66,666$</td><td>$66,666$</td><td rowspan="2">无</td></tr>
<tr><td>$45 \sim 50$</td><td>15</td><td>10^5</td><td>10^5</td><td>10^5</td></tr>
</table>

特殊性质 A：无事件 2；

特殊性质 B：$p_i = i - 1$；

特殊性质 C：$v_i = v = 1$；

NOI2023 省选

DAY2

时间：2023 年 4 月 2 日 08:30 ~ 13:00

题目名称	过河卒	填数游戏	染色数组
题目类型	传统型	传统型	传统型
目录	zu	game	color
可执行文件名	zu	game	color
输入文件名	zu.in	game.in	color.in
输出文件名	zu.out	game.out	color.out
每个测试点时限	1.0 秒	2.0 秒	3.0 秒
内存限制	1 GiB	1 GiB	1 GiB
测试点数目	20	25	20
测试点是否等分	是	是	是

提交源程序文件名

对于 C++ 语言	zu.cpp	game.cpp	color.cpp

编译选项

对于 C++ 语言	-O2 -std=c++14 -static

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考各省的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 全国统一评测时采用的机器配置为：Intel(R) Core(TM) i7-8700K CPU @3.70GHz，内存 32GB。上述时限以此配置为准。
9. 只提供 Linux 格式附加样例文件。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

过河卒（zu）

【题目背景】

棋盘上有一个过河卒，需要走到底线。卒行走的规则是可以向左移动一格，向右移动一格或者向前移动一格。同时在棋盘上有两个另一方的棋子，需要拦截这个卒走到底线。这两个棋子的走法和帅一致，可以走到前后左右四个方向上相邻的格子。因此本题可以称为“帅拦过河卒”。

【题目描述】

有一个 n 行 m 列的棋盘。我们用 (i,j) 表示第 i 行第 j 列的位置。棋盘上有一些障碍，还有一个黑棋子和两个红棋子。

游戏的规则是这样的：红方先走，黑方后走，双方轮流走棋。红方每次可以选择一个红棋子，向棋盘的相邻一格走一步。具体而言，假设红方选择的这个棋子位置在 (i,j)，那么它可以走到 $(i-1,j),(i+1,j),(i,j-1),(i,j+1)$ 中的一个，只要这个目的地在棋盘内且没有障碍且没有红方的另一个棋子。

黑方每次可以将自己的棋子向三个方向之一移动一格。具体地，假设这个黑棋子位置在 (i,j)，那么它可以走到 $(i-1,j),(i,j-1),(i,j+1)$ 这三个格子中的一个，只要这个目的地在棋盘内且没有障碍。

在一方行动之前，如果发生以下情况之一，则立即结束游戏，按照如下的规则判断胜负（列在前面的优先）：

- 黑棋子位于第一行。此时黑方胜。
- 黑棋子和其中一个红棋子在同一个位置上。此时进行上一步移动的玩家胜。
- 当前玩家不能进行任何合法操作。此时对方胜。

现在假设双方采用最优策略，不会进行不利于自己的移动。也就是说：

- 若存在必胜策略，则会选择所有必胜策略中，不论对方如何操作，本方后续获胜所需步数最大值最少的操作。
- 若不存在必胜策略，但存在不论对方如何行动，自己都不会落败的策略，则会选择任意一种不败策略。
- 若不存在不败策略，则会选择在所有策略中，不论对方如何操作，对方后续获胜所需步数最小值最大的操作。

如果在 $100^{100^{100}}$ 个回合之后仍不能分出胜负，则认为游戏平局。请求出游戏结束时双方一共移动了多少步，或者判断游戏平局。

【输入格式】

从文件 ***zu.in*** 中读入数据。

本题有多组测试数据。

输入的第一行包含两个整数 id, T，分别表示测试点编号和数据组数。特别地，样例的 id 为 0。

接下来包含 T 组数据，每组数据的格式如下：

第一行包含两个正整数 n, m，表示棋盘的行数和列数。

接下来 n 行，每行包含一个长度为 m 的字符串，其中第 i 行的第 j 个字符表示棋盘上 (i, j) 这个位置的状态。

在这些字符中：`'.'` 表示空位；`'#'` 表示障碍物；`'X'` 表示黑棋；`'O'` 表示红棋。

保证黑棋恰好有一个，红棋恰好有两个，且黑棋不在第一行。

【输出格式】

输出到文件 ***zu.out*** 中。

对于每组数据，输出一行字符串。

如果游戏平局，请输出一行 `"Tie"`。

如果红方胜，请输出一行 `"Red t"`。其中 `t` 为游戏结束时双方移动的步数之和。显然这应该是一个奇数。

如果黑方胜，请输出一行 `"Black t"`。其中 `t` 为游戏结束时双方移动的步数之和。显然这应该是一个偶数。

注意，请输出双引号内的字符串，不包含双引号。

【样例 1 输入】

```
0 5
4 5
...#O
.#..#
#O#..
.#..X
3 3
#.#
O.O
.X.
3 3
O..
.#X
.O.
5 5
.....
```

```
.....
..O..
#..#.
O#.X.
9 9
...######
.#.......
.#######.
.#.#.....
.#O#.####
.#.#.....
.#######.
.#X......
.O.......
```

【样例 1 输出】

```
Black 0
Black 2
Black 2
Tie
Red 75
```

【样例 1 解释】

第一组数据，红方第一步没有可行的移动，所以黑方胜。

第二组数据，无论第一步红方怎么移动，黑方都可以在下一步让黑棋子与红棋子在同一个位置。

第三组数据，无论第一步红方怎么移动，黑方都可以将自己的棋子往上移动一格来达成胜利。

第四组数据，有一个红棋子不能动。另一个红棋子可以在第三行移动来防止黑棋子进入第一行。黑棋子也可以一直在第五行移动。如果红棋子到达第五行，黑棋子可以选择从另一边逃走。

第五组数据，在最后一行的那个红棋子可以从左边绕一圈抓住黑棋子。注意另一个红棋子可以移动。

【样例 2】

见选手目录下的 ***zu/zu2.in*** 与 ***zu/zu2.ans***。

【样例 2 解释】

这个样例中的每一组数据都满足测试点 5 到 13 中某一个测试点的限制。

【子任务】

对于所有的数据，保证：$1 \le T \le 10$；$2 \le n \le 10$；$1 \le m \le 10$；id 等于测试点编号。

对于每组数据保证：棋盘上的黑棋恰好有一个，红棋恰好有两个，且黑棋不在第一行。

- 测试点 $1 \sim 4$：保证要么平局，要么红方在开始时无法移动。
- 测试点 $5 \sim 6$：保证 $n \ge 4$。保证棋盘上第 $n-1$ 行的每一个格子都是障碍物，且棋盘上其他行没有障碍物。保证黑棋在前 $n-2$ 行，有一个红棋在前 $n-2$ 行，另一个红棋在第 n 行。
- 测试点 $7 \sim 9$：保证 $m = 1$。
- 测试点 $10 \sim 13$：保证要么平局，要么存在策略可以在 9 步之内结束游戏。
- 测试点 $14 \sim 20$：无特殊限制。

填数游戏（game）

【题目描述】

众所周知，Alice 和 Bob 是一对好朋友。今天，他们约好一起玩游戏。

一开始，他们各自有一张空白的纸条。接下来，他们会在纸条上依次写 n 个 $[1,m]$ 范围内的正整数。等 Alice 写完，Bob **在看到 Alice 写的纸条之后开始写他的纸条**。

Alice 需要保证她写下的第 i 个数在集合 S_i 中，Bob 需要保证他写下的第 i 个数在集合 T_i 中。**题目保证** $1 \le |S_i|, |T_i| \le 2$。

Alice 喜欢相同，因此，她希望她写下的数与 Bob 写下的数对应位置相同的个数尽量多。Bob 喜欢不同，因此，他希望他写下的 n 个数 $b_1, \cdots, b_n$ 互不相同。在此基础上，Bob 希望他写下的数与 Alice 写下的数对应位置相同的个数尽量少。

即设 Alice 写下的数为 $a_1, \cdots, a_n$，Bob 写下的数为 $b_1, \cdots, b_n$，记 X 为满足 $1 \le i \le n, a_i = b_i$ 的下标 i 的个数，则

- Alice 希望最大化 X，
- Bob 在**保证** $b_1, \cdots, b_n$ **互不相同的前提**希望最小化 X。

你首先想知道 Bob 能否保证他写下的 n 个数互不相同。如果 Bob 能够做到，你想知道**在双方均采取最优策略的前提下** X 的值会是多少。

【输入格式】

从文件 ***game.in*** 中读入数据。

本题有多组测试数据。

输入的第一行包含一个正整数 T，表示测试数据组数。

接下来包含 T 组数据，每组数据的格式如下：

第一行包含两个正整数 n, m，表示纸条上需要写的数的个数和数的值域。

接下来 n 行，每行输入的第一个整数为 $|S_i|$ 表示集合 S_i 的元素个数，接下来输入 $|S_i|$ 个正整数描述 S_i 中的元素。

接下来 n 行，每行输入的第一个整数为 $|T_i|$ 表示集合 T_i 的元素个数，接下来输入 $|T_i|$ 个正整数描述 T_i 中的元素。

【输出格式】

输出到文件 ***game.out*** 中。

对于每组测试数据输出一行：若 Bob 无法做到他写下的 n 个数互不相同，输出 -1；否则输出在双方均采取最优策略的前提下 X 的值。

【样例 1 输入】

```
1
3 4
1 3
2 1 2
2 3 4
2 1 2
2 2 3
2 3 4
```

【样例 1 输出】

```
1
```

【样例 1 解释】

在这组样例中，$S_1 = \{3\}, S_2 = T_1 = \{1, 2\}, S_3 = T_3 = \{3, 4\}, T_2 = \{2, 3\}$。
Alice 的填法有 4 种，列举如下：
第一种：$a_1 = 3, a_2 = 1, a_3 = 3$。
第二种：$a_1 = 3, a_2 = 1, a_3 = 4$。
第三种：$a_1 = 3, a_2 = 2, a_3 = 3$。
第四种：$a_1 = 3, a_2 = 2, a_3 = 4$。

由于 Bob 必须保证他所填的数互不相同，所以他有以下填法：
第一种：$b_1 = 1, b_2 = 2, b_3 = 3$。
第二种：$b_1 = 2, b_3 = 3, b_3 = 4$。
第三种：$b_1 = 1, b_2 = 2, b_3 = 4$。
第四种：$b_1 = 1, b_2 = 3, b_3 = 4$。

若 Alice 选择第一种填法，则 Bob 为最小化 X，选择第二种填法，得到 $X = 0$。
若 Alice 选择第二种填法，则 Bob 为最小化 X，选择第一种填法，得到 $X = 0$。
若 Alice 选择第三种填法，则 Bob 为最小化 X，选择第二种填法，得到 $X = 0$。
若 Alice 选择第四种填法，则 Bob 无论选择哪种填法，X 均不小于 1。
因此，Alice 为最大化 X 的值，她会选择第四种填法。

【样例 2】

见选手目录下的 ***game/game2.in*** 与 ***game/game2.ans***。

这个样例满足测试点 $1 \sim 5$ 的条件。

【样例 3】

见选手目录下的 ***game/game3.in*** 与 ***game/game3.ans***。
这个样例满足测试点 6 的条件。

【样例 4】

见选手目录下的 ***game/game4.in*** 与 ***game/game4.ans***。
这个样例满足测试点 7 的条件。

【样例 5】

见选手目录下的 ***game/game5.in*** 与 ***game/game5.ans***。
这个样例满足测试点 8 的条件。

【样例 6】

见选手目录下的 ***game/game6.in*** 与 ***game/game6.ans***。
这个样例满足测试点 9 的条件。

【样例 7】

见选手目录下的 ***game/game7.in*** 与 ***game/game7.ans***。
这个样例满足测试点 $10 \sim 11$ 的条件。

【样例 8】

见选手目录下的 ***game/game8.in*** 与 ***game/game8.ans***。
这个样例满足测试点 $12 \sim 13$ 的条件。

【样例 9】

见选手目录下的 ***game/game9.in*** 与 ***game/game9.ans***。
这个样例满足测试点 $23 \sim 25$ 的条件。

【子任务】

对于所有的数据，保证：$1 \le T \le 1000$；$1 \le n, m \le 10^6$；$1 \le |S_i|, |T_i| \le 2$；集合 S_i, T_i 都是 $\{1, 2, \cdots, m\}$ 的子集且集合内元素两两不同。

表格中 $\sum n, \sum m$ 分别表示同个测试点内所有测试数据的 n 总和和 m 总和。$\sum n^2, \sum m^2, \sum n^3, \sum m^3$ 的含义类似。

测试点编号	数据范围	特殊性质
1	$T \le 20, n, m \le 10$	
2		
3		
4		
5		
6	$n, m \le 200, \sum n^3, \sum m^3 \le 4 \cdot 10^7$	A
7		B
8		C
9		D
10		
11		
12	$n, m \le 2000, \sum n^2, \sum m^2 \le 4 \cdot 10^7$	
13		
14	$n, m \le 1.5 \cdot 10^5, \sum n, \sum m \le 3 \cdot 10^5$	A
15		B
16		
17		C
18		
19		D
20		
21		
22		
23	$n, m \le 10^6, \sum n, \sum m \le 1.5 \cdot 10^6$	
24		
25		

特殊性质 A：对于任何 $1 \le i \le n$，S_i 和 T_i 互不相交，即 $S_i \cap T_i = \emptyset$。
特殊性质 B：$n \ge 3$，且对于任何 $1 \le i < n$，$T_i = \{i, i+1\}$，且 $T_n = \{n, 1\}$。
特殊性质 C：对于任何 $1 \le i \le n$，$|S_i| = 1$。
特殊性质 D：对于任何 $1 \le i \le n$，$S_i = T_i$。

【提示】

本题部分测试点读入规模较大，我们建议你采取效率较高的读入方式。

染色数组（color）

【题目描述】

给定一个长度为 n 的正整数数组 A，其中每个数都在 1 到 m 之间，从左到右排成一排。现在要将每个数字染成红色或者绿色，我们定义一个染色方案为优秀的染色方案，当且仅当它满足：

1. 每个数 A_i 要么被染成红色，要么被染成绿色。
2. 红色的数从左到右依次严格递增，绿色的数从左到右依次严格递减。

例如：1 9 3 4 7 6 中，将 1 3 4 7 染成红色，9 6 染成绿色是优秀的染色方案（193476）；1 3 4 6 染成红色，9 7 染成绿色也是优秀的染色方案（193476）。但是将 1 4 7 6 染成红色，9 3 染成绿色则**不是**优秀的染色方案，因为 1 4 7 6 不是递增的。1 9 5 5 中，将 1 和任意一个 5 染色红色，9 和另一个 5 染成绿色，也是优秀的染色方案（其中一种是 1955）。

如果一个数组**至少存在两个不同的**优秀的染色方案，那么称这个数组是**完美**的。（两个染色方案不同当且仅当至少存在一个位置上的数字被染成不同的颜色）。

例如，1 9 3 4 7 6 和 1 9 5 5 都是完美的，因为上面已经分别给出了 2 种优秀的染色方案。而 2 3 3 3 则**不是**完美的，因为找不到任何一种优秀的染色方案。同时 1 5 3 6 4 也不是完美的，因为仅存在一种优秀的染色方案（15364）。

补充说明：如果红色的数只有 0 个或者 1 个，我们也认为它严格递增；同理如果绿色的数只有 0 个或者 1 个，我们也认为它严格递减。例如 123，123都是优秀的染色方案，因此 1 2 3 是完美的数组。

我们定义一种给染色方案打分的方式。

对于每个的有序元素对 $A_i, A_j (i < j)$：

1. 如果 A_j 染成红色，且 $A_j < A_i$，则该元素对得 $m - A_j + 1$ 分；
2. 如果 A_j 染成绿色，且 $A_j > A_i$，则该元素对得 A_j 分；
3. 不满足 1 或 2，则该元素对得 0 分。

则一个染色方案的得分为所有有序元素对的得分和。

一个完美的数组的得分为它所有优秀的染色方案的得分的**最大值**。

现在确定数组 A 的前 t 个数 $A_1, A_2, \ldots, A_t$，你需要回答以下两个问题：

- 第一问：有多少种确定 A 中后 $n - t$ 个数的方案使得 A 是一个完美数组？
- 第二问：所有可能的完美数组的得分和是多少？

由于答案太大，你只需要输出答案在模 998244353 下的结果即可。

【输入格式】

从文件 ***color.in*** 中读入数据。

本题有多组测试数据。

输入的第一行包含一个正整数 C，表示数据组数。

接下来包含 C 组数据，每组数据的格式如下：

第一行包含三个整数 n, m, t，分别表示数组长度，数组内数字的最大值和确定的前缀长度。

第二行包含 t 个正整数 $A_1 \ldots A_t$，表示已经确定的前缀。

【输出格式】

输出到文件 ***color.out*** 中。

对于每组测试数据输出一行包含两个用单个空格隔开的整数。

- 第一个整数表示优秀数组的数量模 998244353 的值；
- 第二个整数表示优秀数组的得分之和模 998244353 的值。

【样例 1 输入】

```
5
6 10 6
1 9 3 4 7 6
5 8 4
1 7 2 6
9 10 2
3 6
6 11 6
1 7 5 8 3 9
9 10 5
5 10 6 4 7
```

【样例 1 输出】

```
1 63
8 245
29378 1267731
1 17
78 1820
```

【样例 2】

见选手目录下的 ***color/color2.in*** 与 ***color/color2.ans***。

【评分方式】

每个测试点 5 分。

每一行应按顺序输出两问的答案，不符合输出格式的输出得 0 分。

程序仅回答对第一问得 1 分，仅回答对第二问得 4 分，两问都答对得 5 分。

如果你不回答第一问或第二问，也需要在对应位置上输出任意一个整数以满足输出格式。

【子任务】

对于所有的数据，保证：$1 \le C \le 5$；$2 \le n \le 50$；$1 \le t \le n$；$1 \le m \le 200$；$1 \le A_i \le m$。

测试点编号	n	m	t
$1 \sim 2$	≤ 50	≤ 200	$= n$
$3 \sim 5$	≤ 8	≤ 200	无特殊限制
$6 \sim 7$	≤ 20	≤ 50	无特殊限制
$8 \sim 10$	≤ 25	≤ 70	无特殊限制
$11 \sim 13$	≤ 35	≤ 100	无特殊限制
$14 \sim 16$	≤ 50	≤ 200	$= 1$
$17 \sim 20$	≤ 50	≤ 200	无特殊限制